

Операционные системы

Лекция 11

Управление памятью

План лекции

- Функции ОС по управлению памятью
- Типы адресов
- Виртуальное адресное пространство
- Способы структуризации виртуального адресного пространства
- Подходы к преобразованию виртуальных адресов в физические
- Классификация методов распределения памяти

Управление памятью

Вот и настало время затронуть второй после процессора по важности и интенсивности использования компонент ЭВМ, а именно – **память**

Память является важнейшим ресурсом, требующим тщательного управления со стороны мультипрограммной операционной системы. Под **памятью** в рамках лекций подразумевается оперативная память компьютера

В отличие от памяти жесткого диска, которую называют **внешней памятью**, оперативной памяти для сохранения информации требуется постоянное электропитание

Особая роль памяти объясняется тем, что процессор может выполнять инструкции программы только в том случае, если они находятся в памяти. Память распределяется как между модулями прикладных программ, так и между модулями самой операционной системы

Управление памятью

В ранних ОС управление памятью сводилось просто к загрузке программы и ее данных с некоторого внешнего накопителя (перфоленты, магнитной ленты или магнитного диска) в память

С появлением мультипрограммирования перед ОС были поставлены новые задачи, связанные с распределением имеющейся памяти между несколькими одновременно выполняющимися программами

Управление памятью

Функциями ОС по управлению памятью в мультипрограммной системе являются:

- **Отслеживание свободной и занятой памяти**
- **Выделение памяти** процессам и **освобождение памяти** при завершении процессов
- **Вытеснение кодов и данных** процессов из оперативной памяти на диск (полное или частичное), когда размеры основной памяти недостаточны для размещения в ней всех процессов, и **возвращение их в оперативную память**, когда в ней освобождается место
- **Настройка адресов** программы на конкретную область физической памяти
- **Защита памяти** – запрет выполняемому процессу записывать данные в память, назначенной другому процессу, или читать их оттуда. Эта функция, как правило, реализуется программными модулями ОС в тесном взаимодействии с аппаратными средствами

Управление памятью

Помимо первоначального выделения памяти процессам при их создании, ОС должна также заниматься **динамическим распределением памяти**, то есть выполнять запросы приложений во время выполнения на выделение им дополнительной памяти

После того как приложение перестает нуждаться в дополнительной памяти, оно может вернуть ее системе

Выделение памяти случайной длины в случайные моменты времени из общего пула памяти приводит к фрагментации и, вследствие этого, к неэффективному ее использованию

Дефрагментация памяти тоже является функцией операционной системы

Управление памятью

Во время работы операционной системы ей часто приходится создавать новые служебные информационные структуры, такие как описатели процессов и потоков, различные таблицы распределения ресурсов, буферы, используемые процессами для обмена данными, синхронизирующие объекты и т. п.

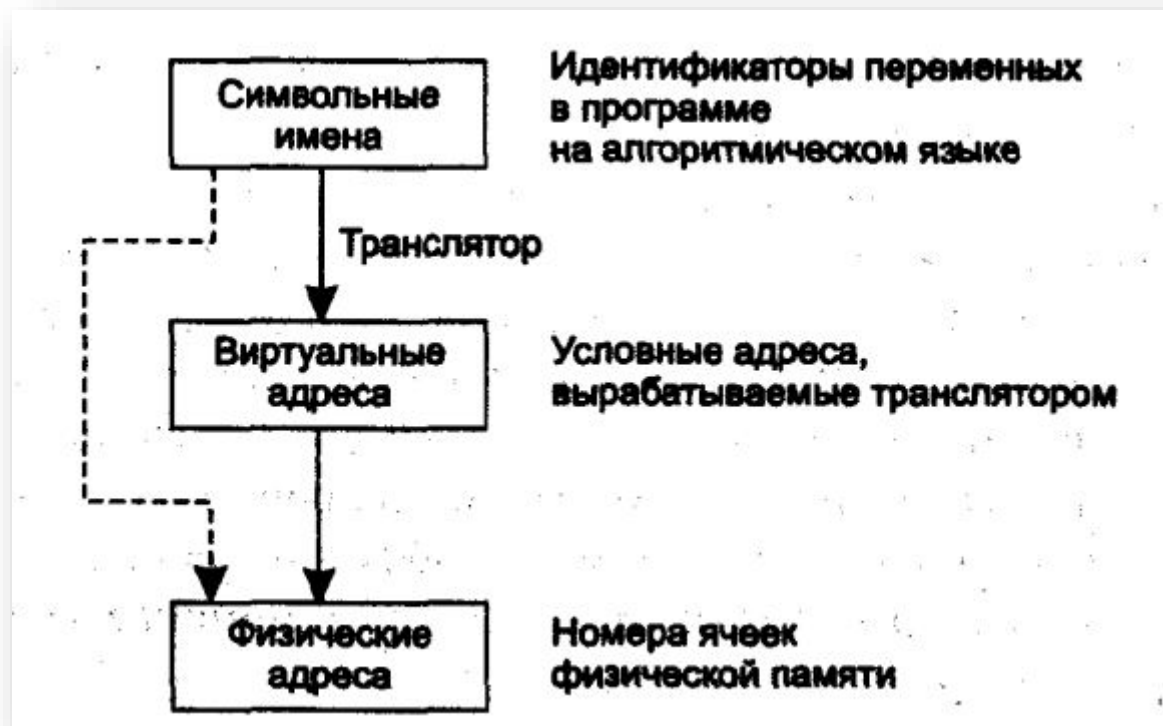
Все эти системные объекты требуют памяти

В некоторых ОС заранее (во время установки) резервируется некоторый фиксированный объем памяти для системных нужд

В других же ОС используется более гибкий подход, при котором память для системных целей выделяется динамически. В таком случае разные подсистемы ОС при создании своих таблиц, объектов, структур и т. п. обращаются к **подсистеме управления памятью** с запросами (также известной как **диспетчер памяти**)

Типы адресов

Для идентификации переменных и команд на разных этапах жизненного цикла программы используются **символьные имена (метки), виртуальные адреса и физические адреса**



Типы адресов

Символьные имена (метки) присваивает пользователь при написании программы на алгоритмическом языке или ассемблере

Виртуальные, математические, или логические, адреса вырабатывает транслятор, переводящий программу на машинный язык

Поскольку во время трансляции в общем случае неизвестно, в какое место оперативной памяти будет загружена программа, то транслятор присваивает переменным и командам условные виртуальные адреса, обычно считая по умолчанию, что начальным адресом программы будет нулевой адрес

Типы адресов

Физические адреса соответствуют номерам ячеек оперативной памяти, где в действительности расположены или будут расположены переменные и команды

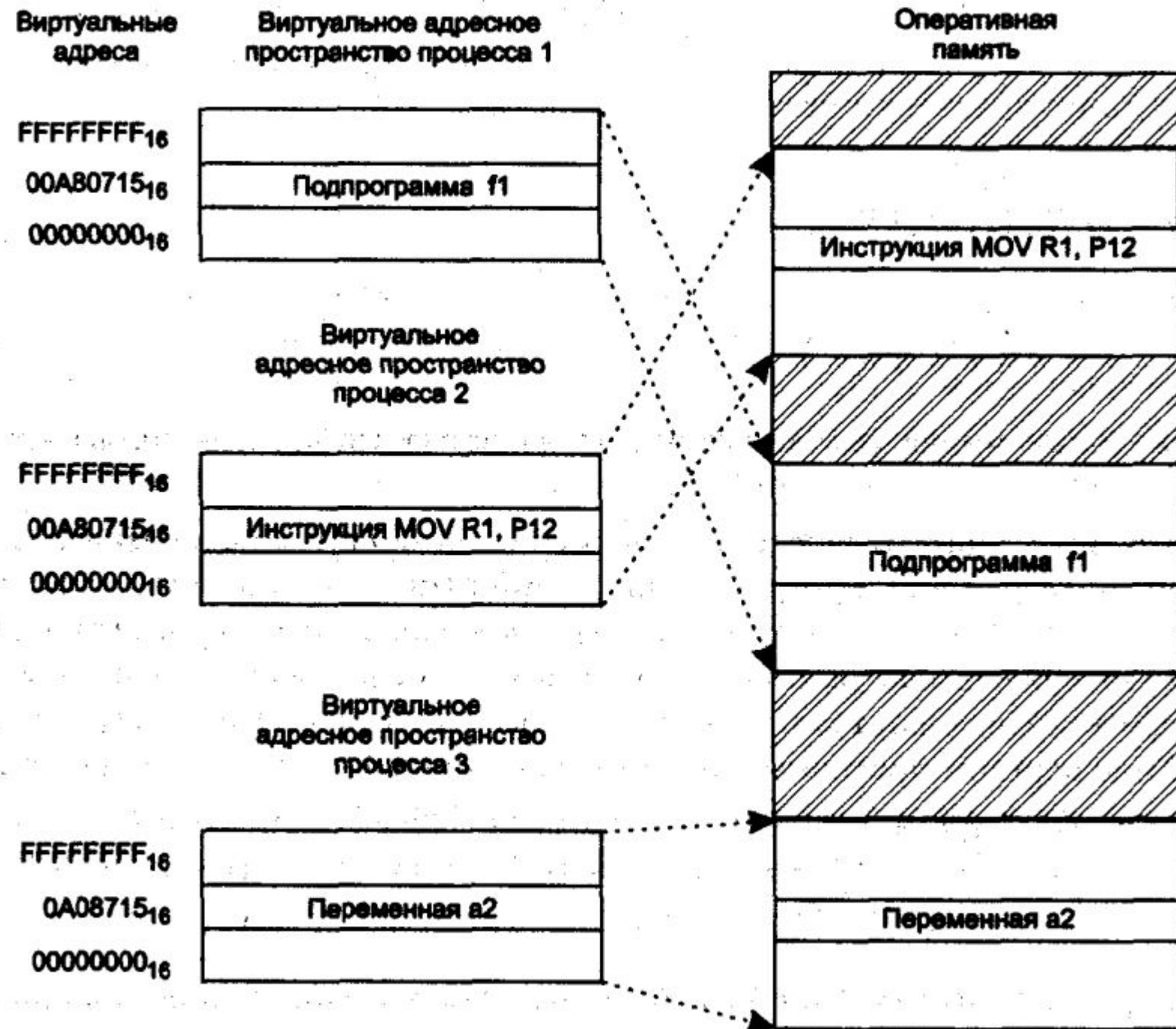
Совокупность виртуальных адресов процесса называется **виртуальным адресным пространством**

Диапазон возможных адресов виртуального пространства у всех процессов является одним и тем же

Например, при использовании 32-разрядных виртуальных адресов этот диапазон задается границами 00000000_{16} и $FFFFFFFF_{16}$

Виртуальное адресное пространство

В то же время каждый процесс имеет **собственное** виртуальное адресное пространство – транслятор присваивает виртуальные адреса переменным и кодам каждой программе независимо



Типы адресов

Преобразование линейного адреса процесса в виртуальный адрес выполняется операционной системой посредством настройки регистров процессора

Обычно линейный адрес процесса отличается от виртуального адреса только интерпретацией бит в этом адресе

Преобразование виртуального адреса в физический адрес выполняется аппаратным образом, а именно – **процессором**

Виртуальное адресное пространство

Совпадение виртуальных адресов переменных и команд различных процессов не приводит к конфликтам, так как в том случае, когда эти переменные одновременно присутствуют в памяти, операционная система отображает их на разные физические адреса

Если необходимо, чтобы несколько процессов разделяли общие данные или коды, операционная система отображает соответствующие участки виртуального адресного пространства этих процессов на один и тот же участок физической памяти (вы с ними уже знакомы с такими механизмами, которые используют общую память, это ваши любимые файлы отображенные в память)

Способы структуризации

В разных операционных системах используются разные способы структуризации виртуального адресного пространства

В одних ОС виртуальное адресное пространство процесса подобно физической памяти представлено в виде непрерывной последовательности виртуальных адресов

Такую структуру адресного пространства называют **линейной**, или **плоской** (flat)

При этом виртуальным адресом является единственное число, представляющее собой смещение относительно начала (обычно это значение 000...000) виртуального адресного пространства (а)

Адрес такого типа называют **линейным виртуальным адресом**

Способы структуризации

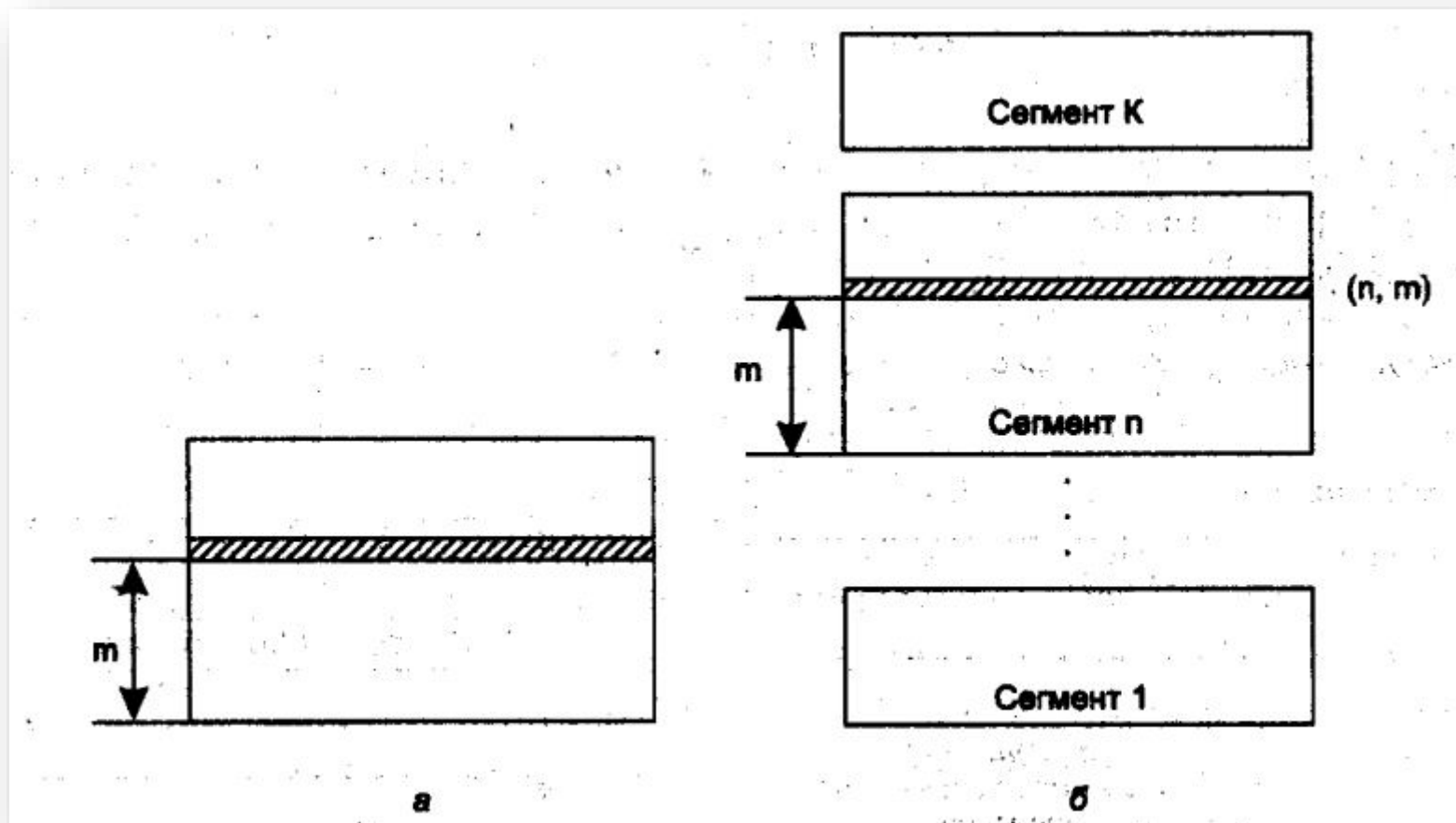
В других ОС виртуальное адресное пространство делится на части, называемые **сегментами** (или секциями, или областями, или другими терминами)

В этом случае говорят, что виртуальное адресное пространство имеет **сегментированную структуру**

Для позиционирования данных здесь помимо линейного виртуального адреса может быть использован виртуальный адрес, учитывающий разбиение на сегменты и представляющий собой пару чисел (n, m) , где n – определяет сегмент, а m – смещение внутри сегмента (б)

Существуют и более сложные способы структуризации виртуального адресного пространства, когда виртуальный адрес образуется тремя или даже более числами

Способы структуризации



Подходы к преобразованию адресов

Задачей операционной системы является отображение индивидуальных виртуальных адресных пространств всех одновременно выполняющихся процессов на общую физическую память

При этом ОС отображает либо все виртуальное адресное пространство, либо только определенную его часть

Процедура преобразования виртуальных адресов в физические должна быть максимально прозрачна для пользователя и программиста

Подходы к преобразованию адресов

Существует два принципиально отличающихся подхода к преобразованию виртуальных адресов в физические

В первом случае замена виртуальных адресов физическими выполняется один раз для каждого процесса во время начальной загрузки программы в память

Специальная системная программа – **перемещающий загрузчик** – на основании имеющихся у нее исходных данных о начальном адресе физической памяти, в которую предстоит загружать программу, а также информации, предоставленной транслятором об адресно-зависимых элементах программы, выполняет загрузку программы, совмещая ее с заменой виртуальных адресов физическими

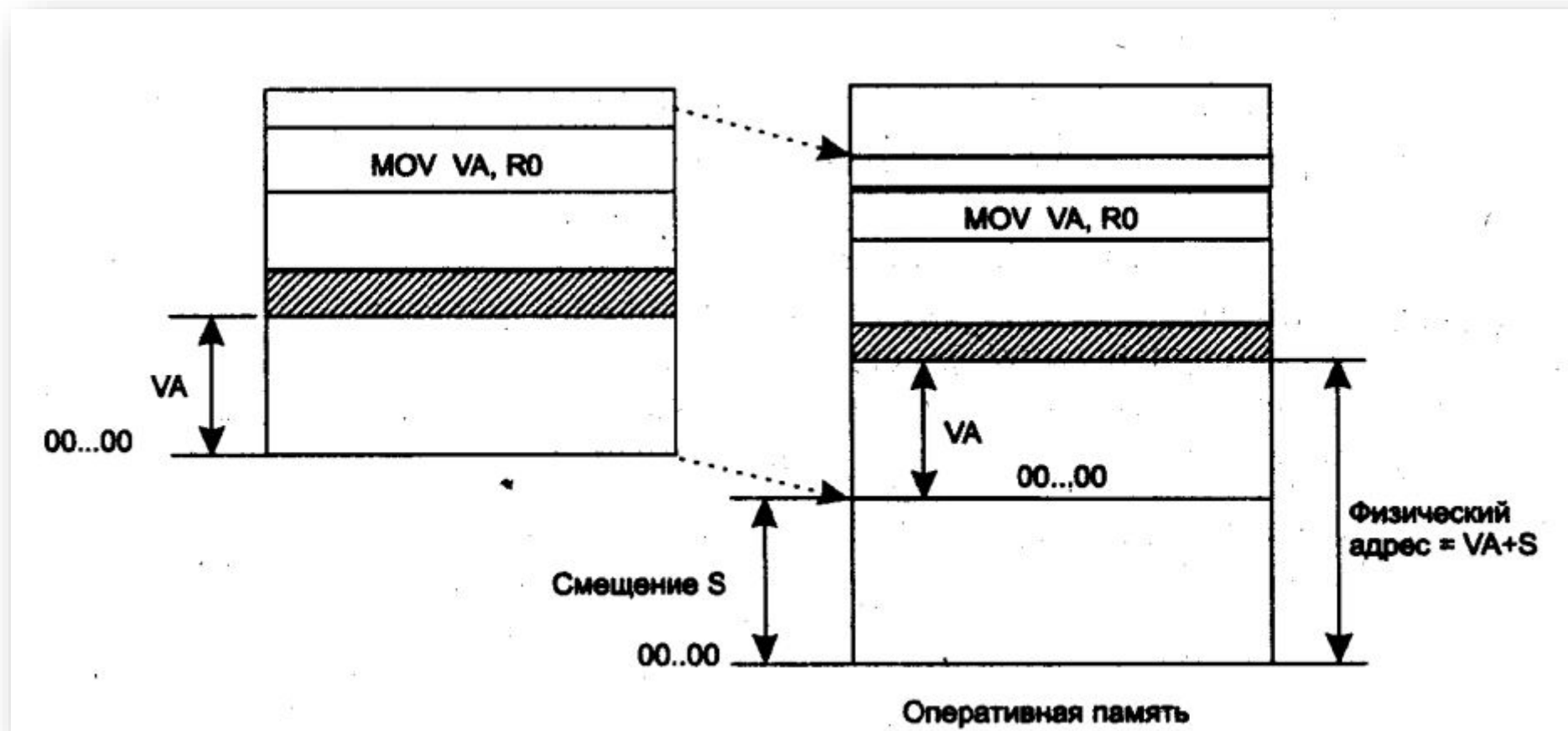
Подходы к преобразованию адресов

Второй способ заключается в том, что программа загружается в память в неизмененном виде в виртуальных адресах, то есть операнды инструкций и адреса переходов имеют те значения, которые выработал транслятор

В наиболее простом случае, когда виртуальная и физическая память процесса представляют собой единые непрерывные области адресов, операционная система выполняет преобразование виртуальных адресов в физические по следующей схеме:

- При загрузке операционная система фиксирует смещение действительного расположения программного кода относительно виртуального адресного пространства
- Во время выполнения программы при каждом обращении к оперативной памяти выполняется преобразование виртуального адреса в физический

Подходы к преобразованию адресов



Подходы к преобразованию адресов

Последний способ является более гибким: в то время как перемещающий загрузчик жестко привязывает программу к первоначально выделенному ей участку памяти, динамическое преобразование виртуальных адресов позволяет перемещать программный код процесса в течение всего периода его выполнения

Однако использование перемещающего загрузчика более экономично, так как в этом случае преобразование каждого виртуального адреса происходит только один раз во время загрузки, а при динамическом преобразовании – при каждом обращении по данному адресу

В некоторых случаях (обычно в специализированных системах), когда заранее точно известно, в какой области оперативной памяти будет выполняться программа, транслятор выдает исполняемый код сразу в физических адресах

Подходы к преобразованию адресов

Необходимо различать **максимально возможное** виртуальное адресное пространство процесса и **назначенное (выделенное)** процессу виртуальное адресное пространство

В первом случае речь идет о максимальном размере виртуального адресного пространства, определяемом архитектурой компьютера, на котором работает ОС, и, в частности, разрядностью его схем адресации (32-разрядная, 64-разрядная и т. п.)

Например, при работе на компьютерах с 32-разрядными процессорами Intel Pentium операционная система может предоставить каждому процессу виртуальное адресное пространство размером до 4 Гбайт (2^{32})

Однако это значение представляет собой только потенциально возможный размер виртуального адресного пространства, который редко на практике бывает необходим процессу

Процесс использует только часть доступного ему виртуального адресного пространства

Подходы к преобразованию адресов

Назначенное виртуальное адресное пространство

представляет собой набор виртуальных адресов, действительно нужных процессу для работы

Эти адреса первоначально назначает программе транслятор на основании текста программы, когда создает кодовый (текстовый) сегмент, а также сегмент или сегменты данных, с которыми работает программа

Затем при создании процесса ОС фиксирует назначенное виртуальное адресное пространство в своих системных таблицах. В ходе своего выполнения процесс может увеличить размер первоначального назначенного виртуального ему адресного пространства, запросив у ОС создания дополнительных сегментов или увеличения размера существующих

Подходы к преобразованию адресов

В любом случае операционная система обычно следит за корректностью использования процессом виртуальных адресов – процессу не разрешается оперировать виртуальным адресом, выходящим за пределы назначенных ему сегментов

Максимальный размер виртуального адресного пространства ограничивается только разрядностью адреса, присущей данной архитектуре компьютера, и, **как правило, не совпадает с объемом физической памяти**, имеющимся в компьютере

Виртуальная память

Сегодня для машин универсального назначения типична ситуация, когда объем виртуального адресного пространства превышает доступный объем оперативной памяти

В таком случае операционная система для хранения данных виртуального адресного пространства процесса, не помещающихся в оперативную память, использует внешнюю память, которая в современных компьютерах представлена жесткими дисками (а)

Именно на этом принципе основана **виртуальная память** – наиболее совершенный механизм, используемый в операционных системах для управления памятью

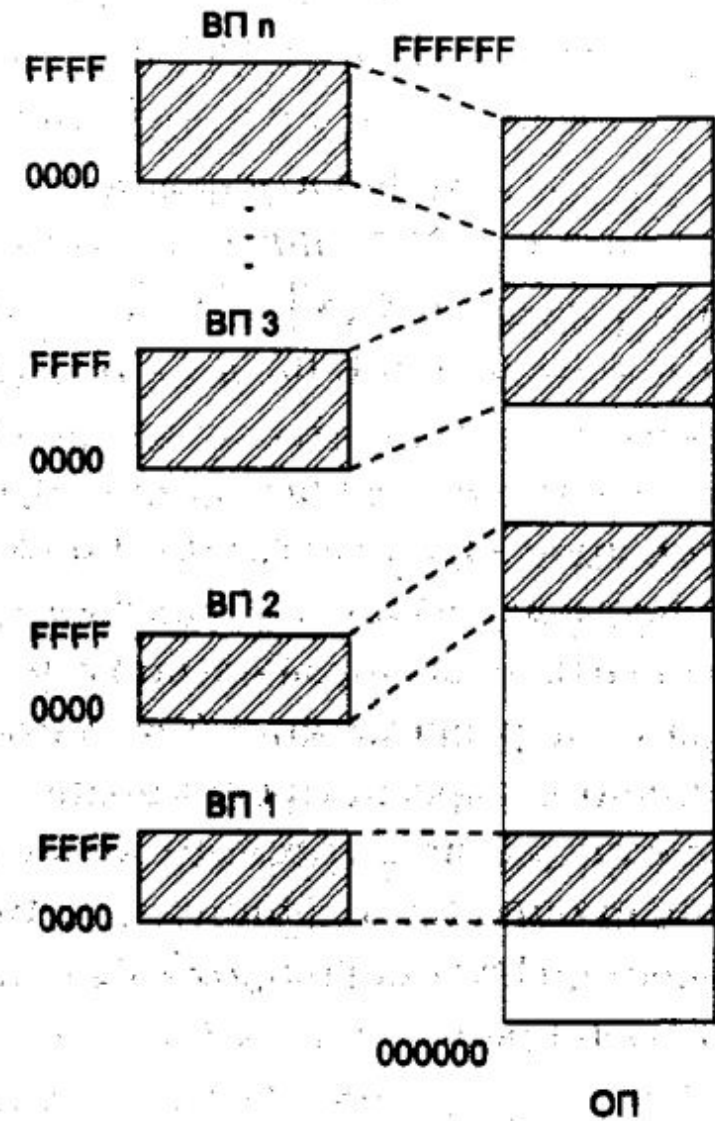
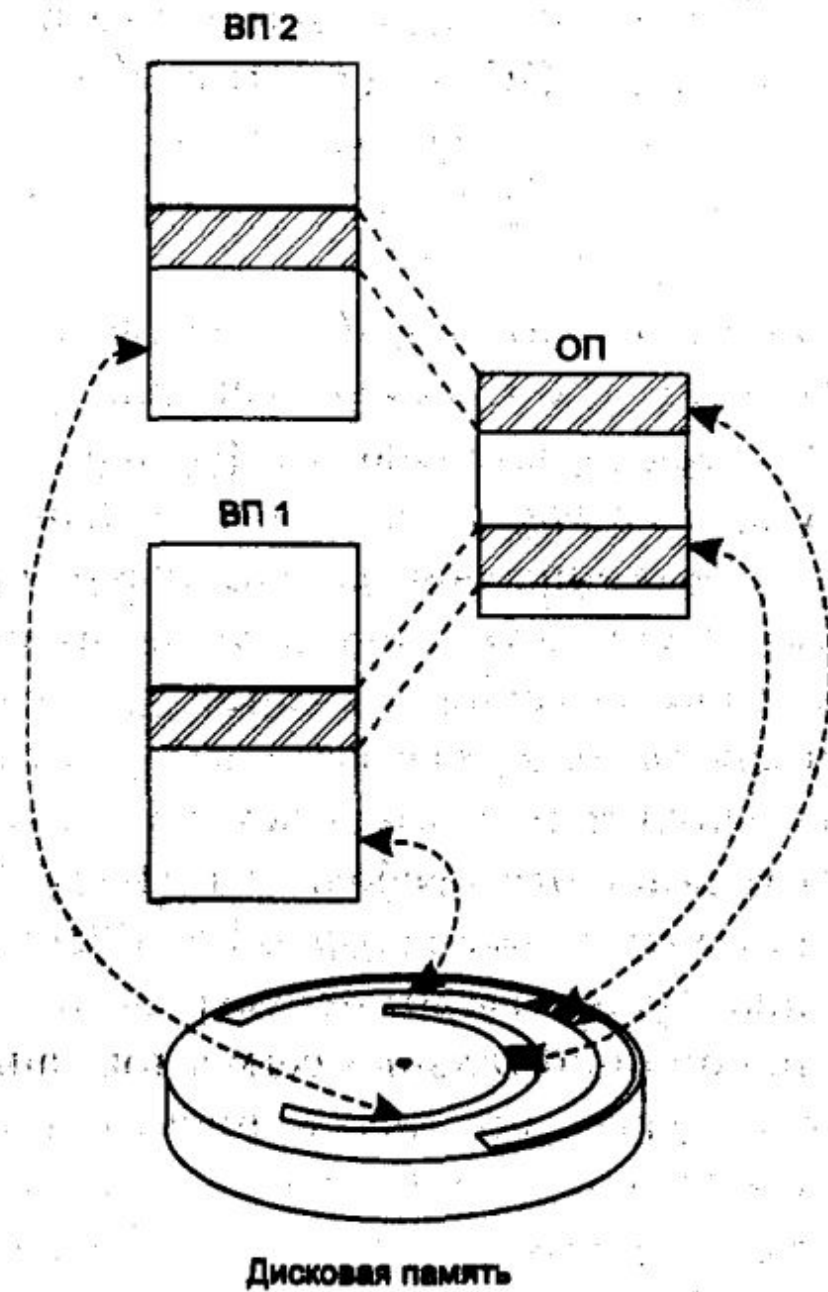
Виртуальная память

Файлы во внешней памяти которые используются для хранения данных виртуальных адресных пространств различных процессов в современных ОС называется **файлом подкачки**

Однако соотношение объемов виртуальной и физической памяти может быть и обратным. Так, в мини-компьютерах 80-х годов разрядности поля адреса нередко не хватало для того, чтобы охватить всю имеющуюся оперативную память

Несколько процессов могло быть загружено в память одновременно и целиком (б)

Виртуальная память



Виртуальное адресное пространство

Во время работы процесса постоянно выполняются переходы от прикладных кодов к кодам ОС, которые вызываются либо явно из прикладных процессов как системные функции, либо как реакция на внешние события или на исключительные ситуации, возникающие при некорректном поведении прикладных кодов

Для того чтобы упростить передачу управления от прикладного кода коду ОС, а также для легкого доступа модулей ОС к прикладным данным (например, для вывода их на внешнее устройство), в большинстве ОС ее сегменты разделяют виртуальное адресное пространство с прикладными сегментами активного процесса

То есть сегменты ОС и сегменты активного процесса образуют единое виртуальное адресное пространство

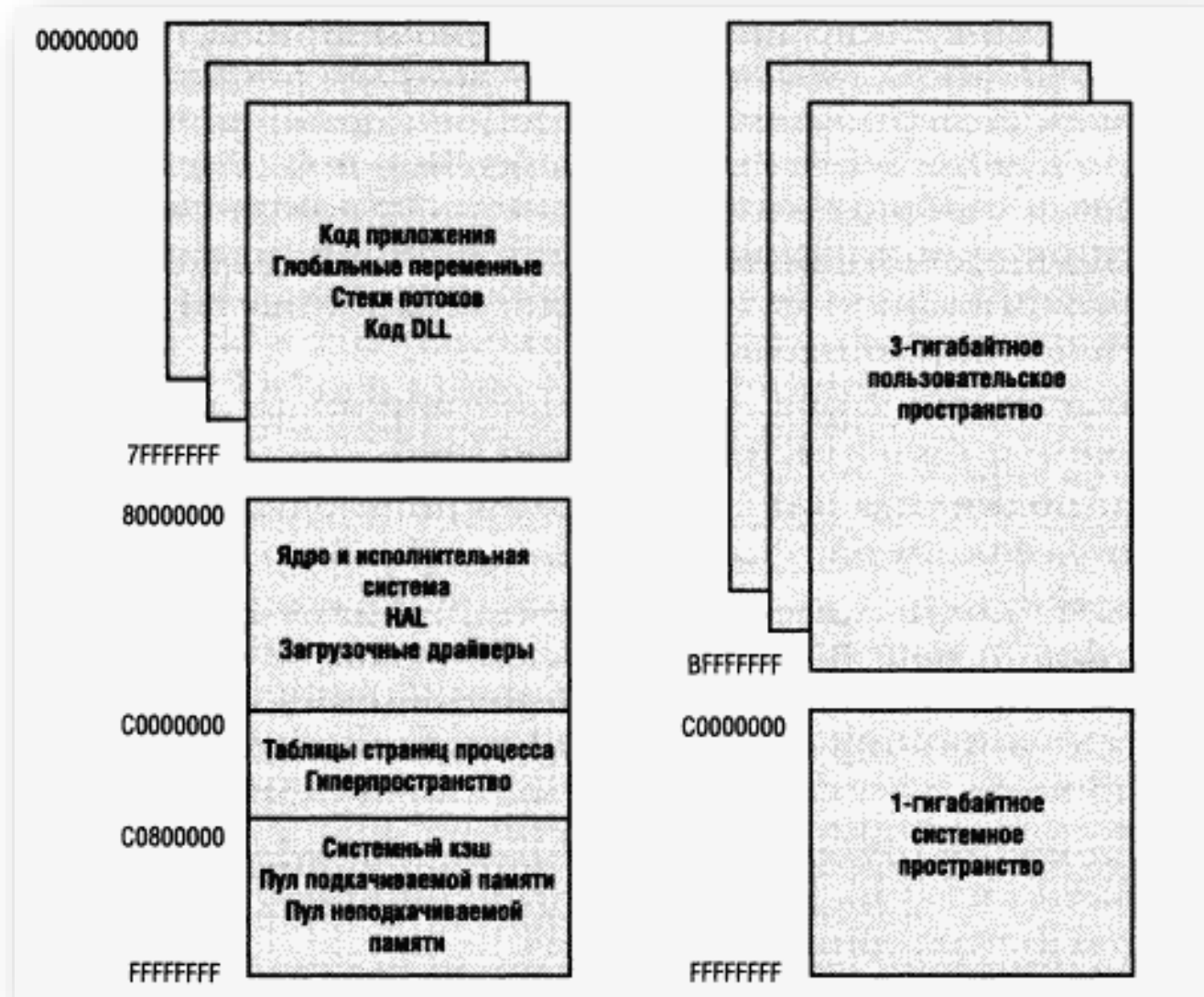
Виртуальное адресное пространство

Обычно виртуальное адресное пространство процесса делится на две непрерывные части: **системную** и **пользовательскую**.

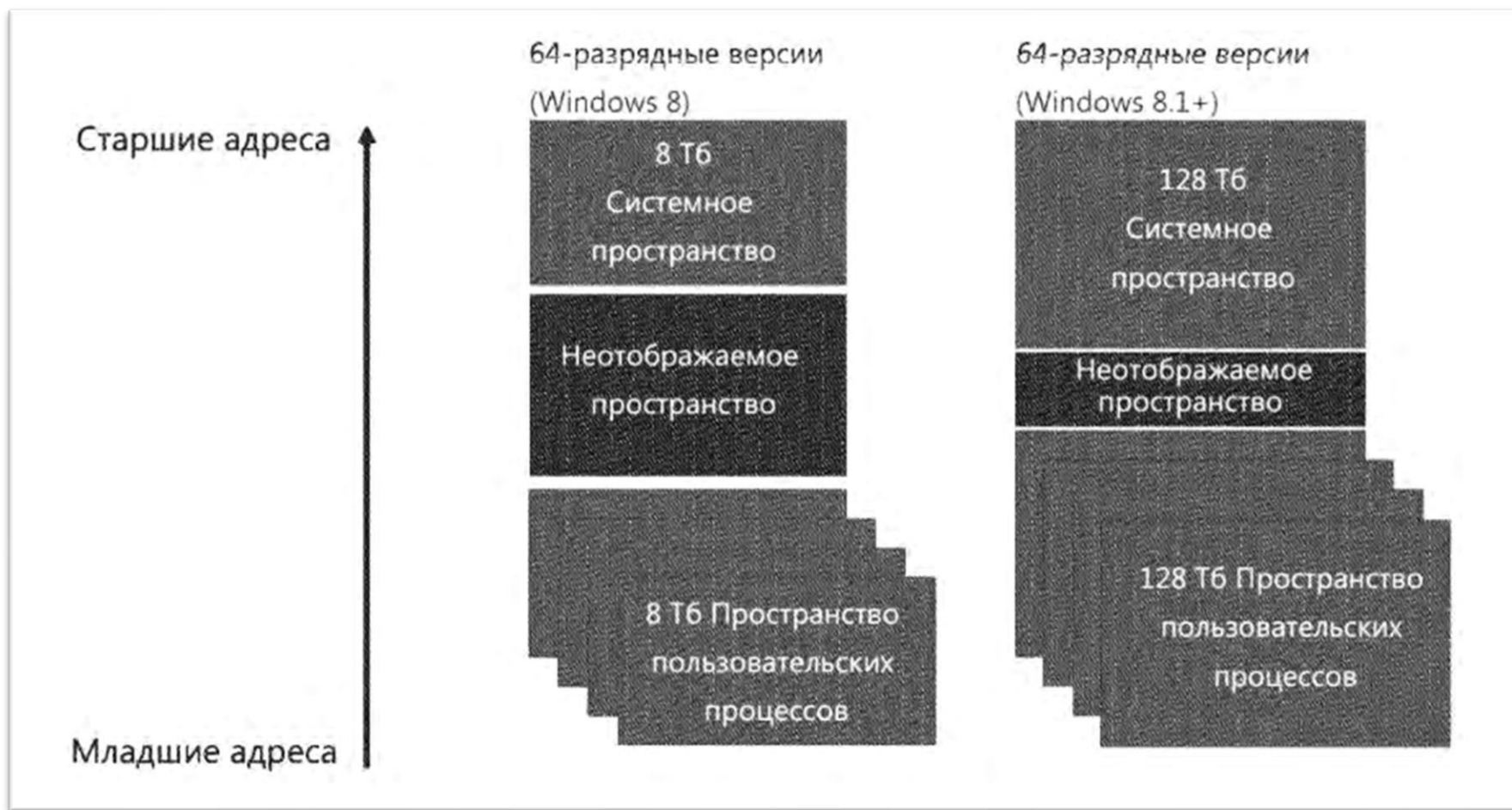
В некоторых ОС (например, ОС семейства Windows NT, OS/2) эти части имеют одинаковый размер – по 2 ГиБ, хотя в принципе деление может и другим, например, 1 ГиБ – для ОС и 3 ГиБ – для прикладных программ

Деление виртуального адресного пространства на две непрерывные области не является обязательным – можно представить себе ОС, которая обходится без него, чередуя свои сегменты и сегменты прикладных программ и размещая их в виртуальном адресном пространстве в произвольном порядке

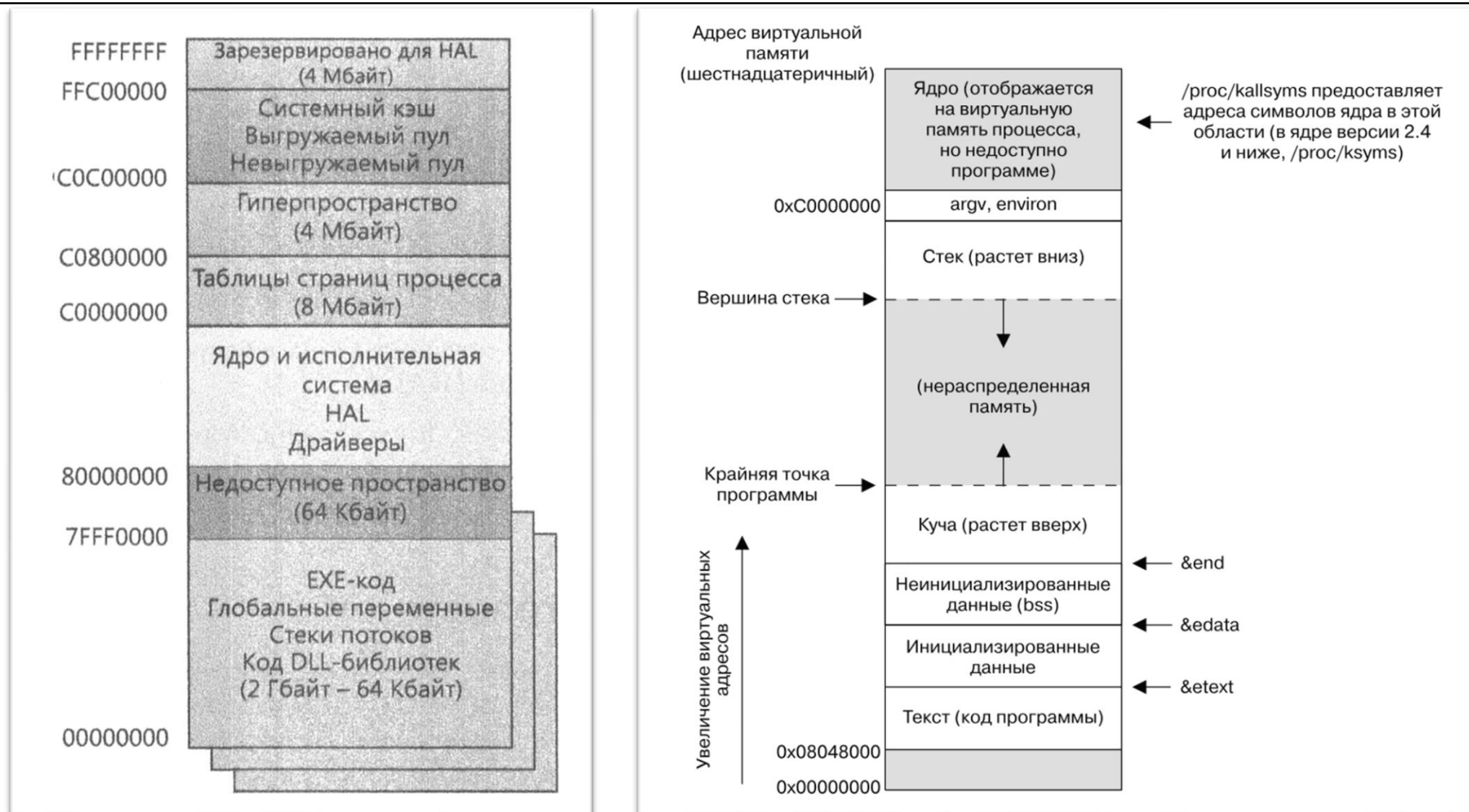
Виртуальное адресное пространство



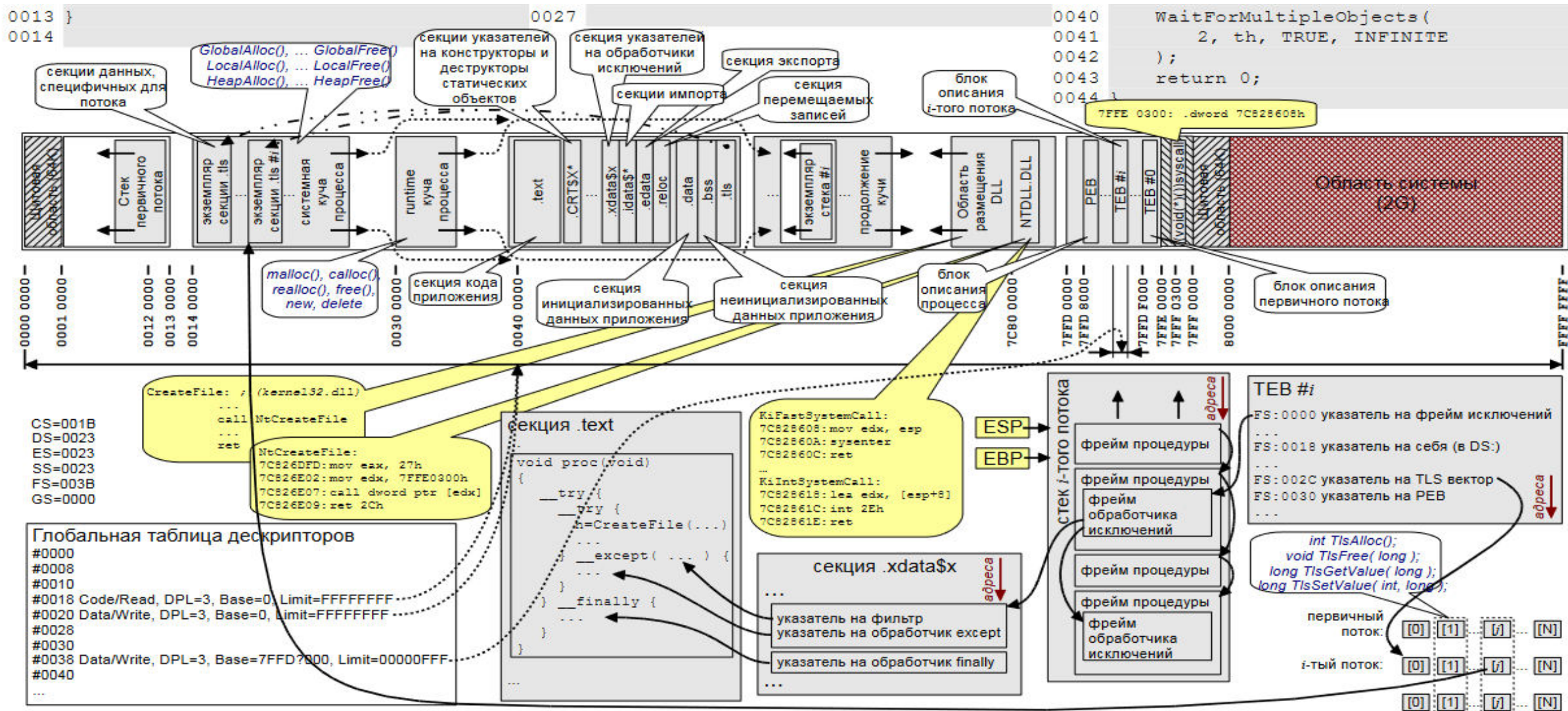
Виртуальное адресное пространство



Виртуальное адресное пространство



Виртуальное адресное пространство



Виртуальное адресное пространство

Виды памяти:

- Куча (уровень пользователя)
 - По умолчанию
 - Общая куча, используемая для передачи больших аргументов экземпляру сеанса Csrss.exe процесса (Windows only)
 - Созданная библиотекой времени выполнения языка C
- Стек (уровень пользователя и ядра)
- Свободная виртуальная память (уровень пользователя)
- Пул памяти (уровень ядра)
 - Выгружаемый
 - Невыгружаемый
- Резервные списки (уровень ядра)

Виртуальное адресное пространство

Куча – это регион зарезервированного адресного пространства. Первоначально большей его части физическая память не передается. По мере того, как программа занимает эту область под данные, специальный диспетчер, управляющий кучами (heap manager), постранично передает ей физическую память (из страничного файла). А при освобождении блоков в куче диспетчер возвращает системе соответствующие страницы физической памяти

Динамическая память, или **куча** – область, из которой память (для переменных) может динамически выделяться в ходе выполнения программы. Верхний конец кучи называют **крайней точкой программы**

Виртуальное адресное пространство

Программный поток должен иметь доступ к временной области памяти для хранения параметров функций, локальных переменных и адреса возврата после вызова функции. Эта часть памяти называется **стеком (stack)**

Стек является довольно статичной частью адресного пространства, в том смысле, что практически не существует инструментов взаимодействия с ним на пользовательском уровне

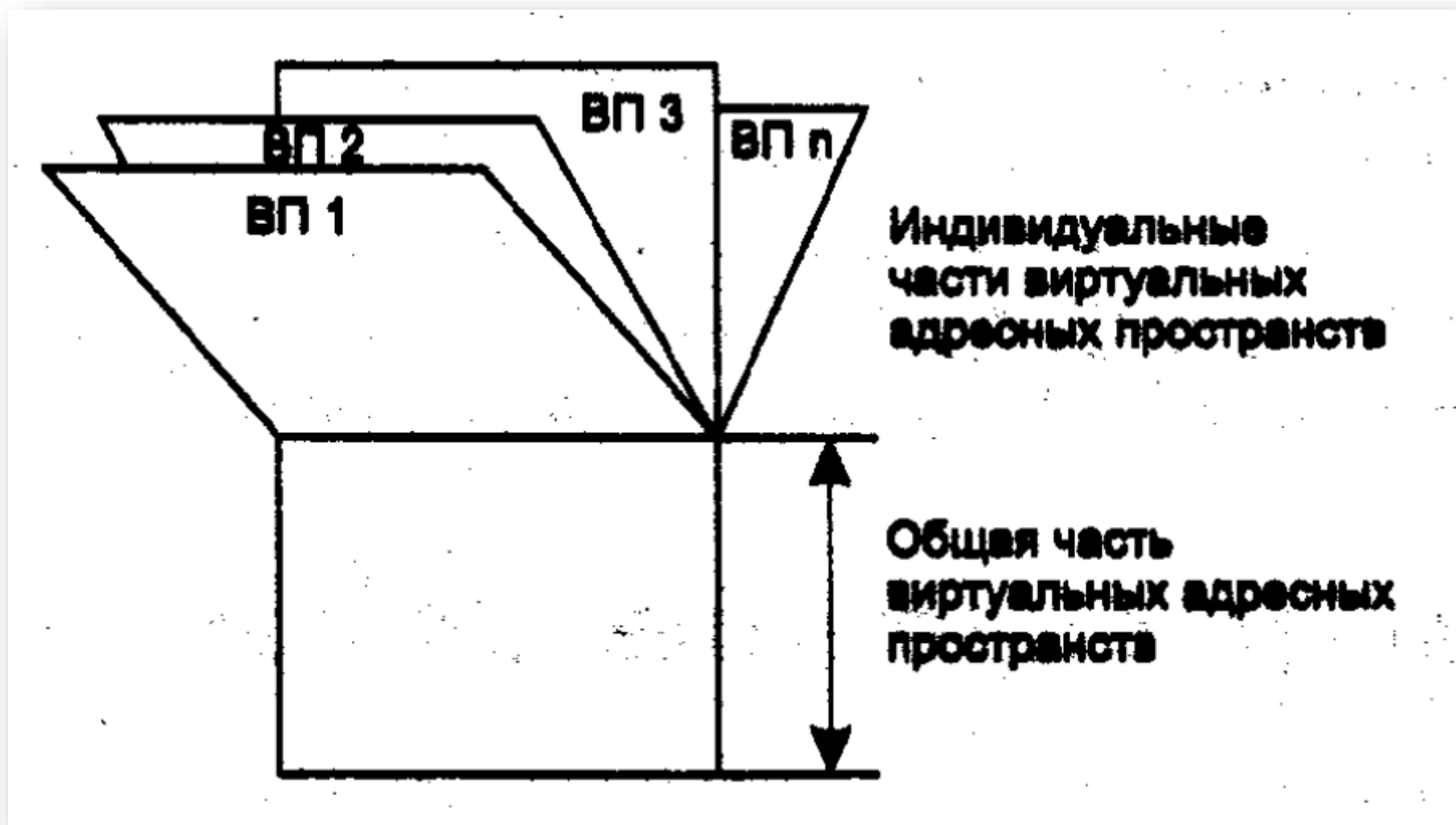
Виртуальное адресное пространство

Часть виртуального адресного пространства каждого процесса, отводимая под сегменты ОС, является **идентичной для всех процессов**

Поэтому при смене активного процесса заменяется только вторая часть виртуального адресного пространства, содержащая его индивидуальные сегменты, как правило, коды и данные прикладной программы

Архитектура современных процессоров отражает эту особенность структуры виртуального адресного пространства, например, в процессорах Intel Pentium существует два типа системных таблиц: одна – для описания сегментов, общих для всех процессов, а другая – для описания индивидуальных сегментов данного процесса. При смене процесса первая таблица остается неизменной, а вторая заменяется новой

Виртуальное адресное пространство



Виртуальная память

Описанное назначение двух частей виртуального адресного пространства – для сегментов ОС и для сегментов прикладной программы – является типичным, но не абсолютным

Имеются и исключения из общего правила. В некоторых ОС существуют системные процессы, порожденные для решения внутренних задач ОС. В этих процессах отсутствуют сегменты прикладной программы, но они могут расположить некоторые свои сегменты (сегменты ОС) в общей части виртуального адресного пространства, а некоторые – в индивидуальной части, обычно предназначенной для прикладных сегментов

И, наоборот, в общей, системной части виртуального адресного пространства размещаются сегменты прикладного кода, предназначенные для совместного использования несколькими прикладными процессами

Виртуальная память

Механизм страничной памяти в большинстве универсальных операционных систем применяется ко всем сегментам пользовательской части виртуального адресного пространства процесса

Исключения могут составлять специализированные ОС, например ОС реального времени, в которых некоторые сегменты жестко фиксируются в оперативной памяти и, соответственно, никогда не выгружаются на диск – это обеспечивает быструю реакцию определенных приложений на внешние события

Виртуальная память

Системная часть виртуальной памяти в ОС любого типа включает область, подвергаемую страничному вытеснению (**paged**) – **выгружаемый пул**, и область, на которую страничное вытеснение не распространяется (**non-paged**) – **невыгружаемый пул**

В **невытесняемой** области размещаются модули ОС, требующие быстрой реакции и/или постоянного присутствия в памяти, например диспетчер потоков или код, который управляет заменой страниц памяти

Остальные модули ОС подвергаются страничному вытеснению, как и пользовательские сегменты

Виртуальная память

Обычно аппаратура накладывает свои ограничения на порядок использования виртуального адресного пространства

Некоторые процессоры (например, MIPS) предусматривают для определенной области системной части адресного пространства особые правила отображения на физическую память

При этом виртуальный адрес прямо отображается на физический (последний либо полностью соответствует виртуальному адресу, либо равен его части)

Такая особая область памяти не подвергается страничному вытеснению, и поскольку достаточно трудоемкая процедура преобразования адресов исключается, то доступ к располагаемым здесь кодам и данным осуществляется очень быстро

Алгоритмы распределения памяти

Следует ли назначать каждому процессу одну непрерывную область физической памяти или можно выделять память «кусками»?

Должны ли сегменты программы, загруженные в память, находиться на одном месте в течение всего периода выполнения процесса или можно их время от времени сдвигать?

Что делать, если сегменты программы не помещаются в имеющуюся память?

Алгоритмы распределения памяти

Разные ОС по-разному отвечают на эти и другие базовые вопросы управления памятью

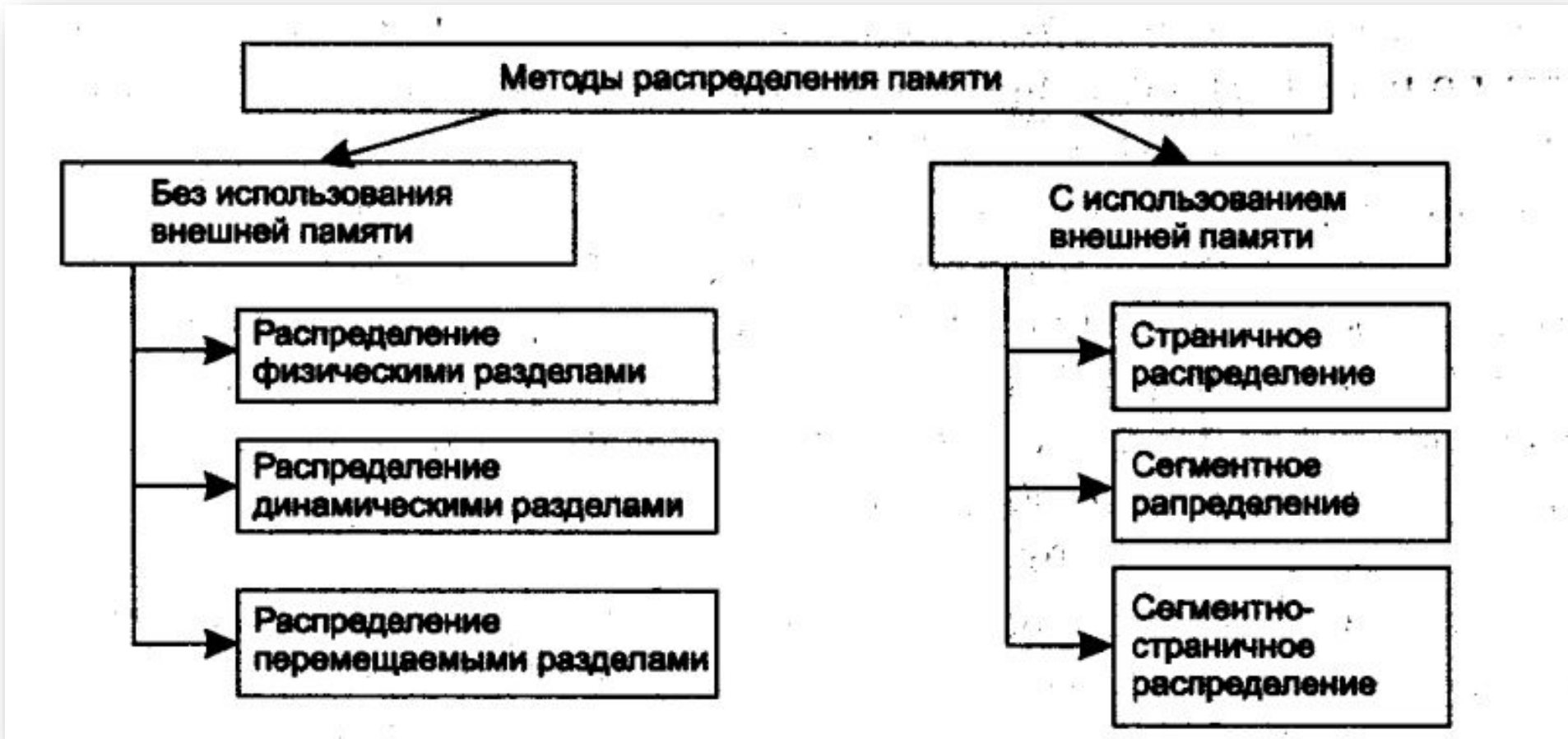
Далее будут рассмотрены наиболее общие подходы к распределению памяти, которые были характерны для разных периодов развития операционных систем

Некоторые из них сохранили актуальность и широко используются в современных ОС, другие же представляют в основном только познавательный интерес, хотя их и сегодня можно встретить в специализированных системах

Все алгоритмы распределения памяти разделены на два класса:

- алгоритмы, в которых используется перемещение сегментов процессов между оперативной памятью и диском
- алгоритмы, в которых внешняя память не привлекается

Алгоритмы распределения памяти



Операционные системы

Лекция 11

Управление памятью